

LAPORAN TUGAS BESAR
MILESTONE II – SYNTAX ANALYSIS
IF2224 TEORI BAHASA FORMAL DAN AUTOMATA
ARION COMPILER



Disusun oleh:

Kelompok 11

“noooRegex” - MMD

Kloce Paul William Saragih	13524040
Suryani Mulia Utami	13524042
Josh Reinhart Zidik	13524048
Syaqina Octavia Rizha	13524088

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025/2026

DAFTAR ISI

BAB 1.....	1
PENDAHULUAN.....	1
BAB 2.....	2
DASAR TEORI.....	2
2.1. Parser.....	2
2.2. Parse Tree.....	2
2.3. Recursive Descent.....	3
BAB 3.....	4
PERANCANGAN.....	4
3.1. Desain Parser.....	4
3.2. Teknologi.....	5
3.3. Struktur Program.....	5
3.4. Fungsi Utama.....	6
3.5. Alur Kerja Program.....	10
3.6. Pendekatan Implementasi.....	11
BAB 4.....	13
HASIL IMPLEMENTASI.....	13
4.1. Kode Program.....	13
4.2. Aturan Grammar.....	17
BAB 5.....	19
EKSPERIMEN.....	19
BAB 6.....	31
KESIMPULAN DAN SARAN.....	31
6.1. Kesimpulan.....	31
6.2. Saran.....	31
REFERENSI.....	32

DAFTAR GAMBAR

Gambar 1.1 Skema Kerja Interpreter.....	1
Gambar 2.2.1 Contoh Parse Tree.....	3
Gambar 3.5.1 Alur Kerja Program.....	10

DAFTAR TABEL

Tabel 3.2.1 Teknologi yang Digunakan.....	5
Tabel 3.4.1 Desain Kelas Utama.....	6
Tabel 5.1 Kasus Uji 1.....	19
Tabel 5.2 Kasus Uji 2.....	20
Tabel 5.3 Kasus Uji 3.....	24
Tabel 5.4 Kasus Uji 4.....	26
Tabel 5.5 Kasus Uji 5.....	29
Tabel 5.6 Kasus Uji 6.....	30

BAB 1

PENDAHULUAN

Bahasa pemrograman digunakan untuk menuliskan instruksi agar komputer dapat menjalankan suatu program tanpa harus menggunakan bahasa mesin secara langsung. Komputer hanya memahami bahasa mesin dalam bentuk biner (0 dan 1), sehingga bahasa pemrograman dikembangkan agar lebih mudah dipahami manusia. Secara umum, bahasa pemrograman dibedakan menjadi low-level yang dekat dengan bahasa mesin dan high-level yang lebih mendekati bahasa manusia. Semakin tinggi tingkat bahasanya, semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.

Salah satu cara untuk menerjemahkan bahasa pemrograman ke dalam kode mesin adalah dengan menggunakan interpreter. Interpreter bekerja dengan membaca, menerjemahkan, dan mengeksekusi kode sumber secara baris demi baris. Pendekatan ini memudahkan dalam menemukan kesalahan karena error dapat langsung diketahui pada baris tertentu. Berbeda dengan compiler yang menerjemahkan seluruh source code menjadi kode mesin (misalnya dalam bentuk object file) sebelum program dijalankan.

Meskipun berbeda dalam cara eksekusi, keduanya tetap melalui tahapan analisis yang serupa. Proses dimulai dari Lexical Analysis yang memecah kode sumber menjadi token-token kecil seperti kata kunci, variabel, dan operator. Selanjutnya, Syntax Analysis yang memeriksa apakah susunan token tersebut sudah sesuai dengan aturan tata bahasa (*grammar*) yang berlaku. Setelah itu, Semantic Analysis memastikan bahwa makna dari kode tersebut benar. Tahap terakhir, Intermediate Code Generation yang mengubah kode ke bentuk perantara yang lebih mudah diproses sebelum akhirnya dieksekusi atau diterjemahkan lebih lanjut menjadi kode mesin.



Gambar 1.1 Skema Kerja Interpreter

Sumber: GeeksForGeeks

BAB 2

DASAR TEORI

2.1. Parser

Syntax Analysis atau Parser bertugas untuk memeriksa urutan token yang dihasilkan oleh lexical analyzer agar sesuai dengan aturan tata bahasa (grammar) dari bahasa pemrograman yang dikompilasi. Parser akan membangun struktur sintaks seperti Parse Tree atau AST (Abstract Syntax Tree) yang merepresentasikan struktur hierarkis program sumber berdasarkan grammar yang telah didefinisikan.

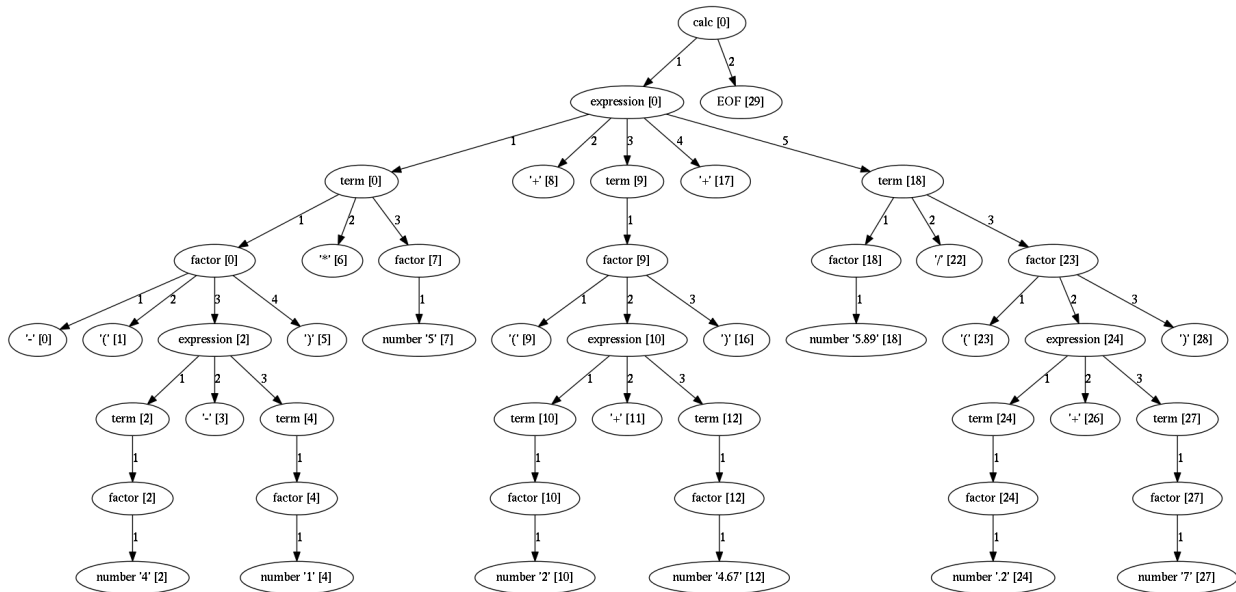
Tahapan Parser bertujuan untuk:

1. Memastikan urutan token membentuk struktur sintaks yang valid.
2. Mendeteksi dan melaporkan kesalahan sintaks (syntax error).
3. Mempersiapkan representasi program untuk tahap berikutnya.

2.2. Parse Tree

Parse Tree adalah struktur data berbentuk pohon yang merepresentasikan derivasi lengkap dari sebuah string input terhadap suatu grammar. Setiap internal node dalam pohon merepresentasikan sebuah simbol non-terminal beserta aturan produksi yang diterapkan, sedangkan setiap leaf node merepresentasikan simbol terminal yang berkorespondensi langsung dengan token dari input. Karena memuat seluruh simbol terminal dan non-terminal tanpa pengecualian, Parse Tree bersifat eksplisit dan lengkap, setiap langkah derivasi grammar tercermin secara langsung dalam strukturnya.

Keunggulan Parse Tree terletak pada ketelusurannya: hubungan antara token input dengan aturan grammar yang menanganinya dapat ditelusuri secara tepat melalui jalur dari leaf ke root. Hal ini berguna untuk keperluan debugging dan verifikasi kebenaran parser. Namun karena memuat semua simbol termasuk yang tidak memiliki kontribusi semantik (seperti tanda kurung, koma, dan kata kunci struktural), Parse Tree cenderung lebih besar dibandingkan AST. Pada tahap selanjutnya, Parse Tree dapat dipangkas menjadi AST dengan menghilangkan node-node yang tidak relevan secara semantik, sehingga lebih efisien untuk diproses oleh tahap semantic analysis dan code generation.



Gambar 2.2.1 Contoh Parse Tree

Sumber: https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

2.3. Recursive Descent

Recursive Descent Parser adalah teknik implementasi top-down parser di mana setiap aturan produksi grammar dipetakan langsung menjadi sebuah fungsi atau prosedur dalam kode program. Parser dimulai dari fungsi yang merepresentasikan simbol awal (start symbol) grammar, kemudian secara rekursif memanggil fungsi-fungsi lain sesuai dengan non-terminal yang muncul dalam aturan produksi yang sedang diproses. Token input dikonsumsi secara sekuensial dari kiri ke kanan, dan keputusan cabang produksi mana yang diambil ditentukan berdasarkan token yang sedang aktif, menjadikannya termasuk kategori parser LL(k) di mana k menyatakan jumlah token lookahead yang diperlukan.

BAB 3

PERANCANGAN

3.1. Desain Parser

Parser dirancang menggunakan metode Recursive Descent Parsing dengan pendekatan *top-down*. Arsitektur ini menerjemahkan aturan grammar menjadi sekumpulan prosedur yang saling berkaitan secara hierarkis.

- **Analisis Top-Down**
Parser bekerja dengan cara mendekomposisi aturan besar menjadi bagian-bagian yang lebih kecil. Proses ini dimulai dari aturan utama program (sebagai simbol non-terminal utama), lalu turun ke bagian detail seperti deklarasi variabel hingga pernyataan perintah. Setiap prosedur bertanggung jawab untuk memastikan urutan token yang dibaca memenuhi kriteria simbol yang ditentukan dalam aturan tersebut.
- **Pengecekan simbol**
Parser akan mencocokkan tipe token yang sedang dibaca dengan simbol terminal yang diharapkan oleh aturan grammar. Jika token sesuai dengan simbol tersebut, maka parser akan lanjut mengambil token berikutnya. Jika tidak sesuai, parser akan menandai bahwa telah terjadi kesalahan sintaks karena ditemukan token yang tidak diharapkan oleh aturan.
- **Backtracking**
Karena beberapa aturan grammar memiliki awalan yang serupa, parser dirancang untuk dapat kembali ke posisi token tertentu setelah mencoba satu jalur aturan. Jika parser gagal memvalidasi suatu jalur, ia akan melakukan reset posisi ke indeks token awal sebelum jalur tersebut dicoba, lalu mencoba jalur alternatif lainnya.
- **Penyusunan Parse Tree**
Setiap kali parser berhasil memvalidasi suatu aturan, akan disusun struktur hierarki yang menyerupai pohon. Pohon ini menggambarkan hubungan antara induk (aturan besar) dan anak (isi dari aturan tersebut/simbol terminal). Hasil akhirnya adalah sebuah gambaran visual yang menunjukkan bagaimana seluruh kode sumber tersusun secara rapi berdasarkan struktur grammar.

- **Error handling**
Jika terdapat kesalahan sintaks, parser akan berhenti, dan mencatat simbol yang diharapkan dan yang ditemukan. Selain itu, parser juga melacak di baris mana kesalahan tersebut terjadi, sehingga pengguna bisa dengan mudah menemukan dan memperbaiki bagian yang salah.

3.2. Teknologi

Berikut adalah teknologi yang digunakan dalam program:

Tabel 3.2.1 Teknologi yang Digunakan

Teknologi yang digunakan	Penjelasan
Bahasa Pemrograman C++ dan STL Library	Digunakan untuk mengimplementasikan lexer, DFA, parser, dan seluruh logika program. Program ini menggunakan library standar C++ seperti fstream untuk pengolahan file; map, set, dan vector untuk pengelolaan data, serta memory untuk manajemen node parse tree secara otomatis.
Build Tool Makefile	Digunakan untuk mengatur proses kompilasi program secara otomatis.

3.3. Struktur Program

Berikut adalah struktur dari program:

```

MMD-TUBES-IF2224-2026/
|
|— doc/
|   |— Laporan-1-MMD.pdf
|   |— Laporan-2-MMD.pdf
|
|— src/
|   |— Lexer/
|       |— dfa_rules.txt
|       |— DFA.cpp

```

```

├── DFA.hpp
├── lexer.cpp
├── lexer.hpp
├── main.cpp
├── Parser.cpp
├── Parser.hpp
├── ParserTree.cpp
├── ParserTree.hpp
├── Symbol.cpp
├── Symbol.hpp
├── Token.cpp
├── Token.hpp
├── test/
│   ├── milestone-1/
│   └── milestone-2/
├── README.md
└── Makefile

```

3.4. Fungsi Utama

Tabel 3.4.1 Desain Kelas Utama

Nama Kelas	Data/Method	Penjelasan
Parser	<code>Parser(std::vector<Token> tokens)</code>	Konstruktor kelas yang menginisialisasi objek Parser dengan list token hasil lexer. Atribut <code>currentToken</code> diset ke 0 sebagai posisi awal token yang akan diproses. Atribut <code>received</code> (pointer ke Token) dan <code>expected</code> (nilai Symbol) digunakan secara internal untuk menyimpan informasi token yang diterima dan yang diharapkan saat terjadi syntax error.
	<code>std::shared_ptr<TreeNode> buildTree()</code>	Method utama yang memulai proses parsing dengan memanggil <code>programNode()</code> sebagai root grammar. Apabila parsing gagal (mengembalikan <code>nullptr</code>), method ini mencetak pesan error ke konsol yang

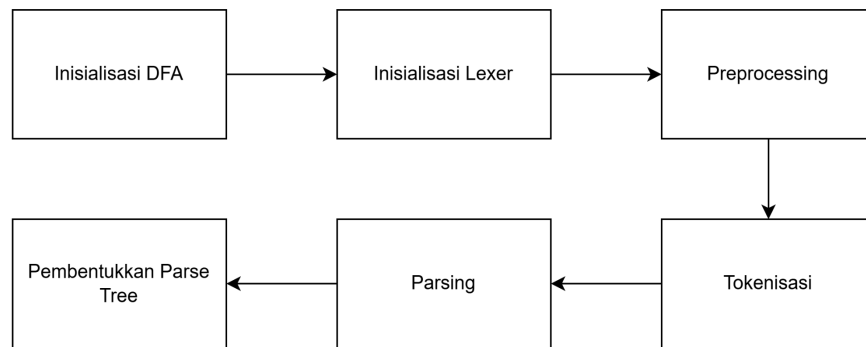
		mencakup simbol yang diharapkan (expected), token yang diterima (received), dan nomor baris terjadinya error.
	<pre>std::shared_ptr<TreeNode> terminal(Symbol symbol)</pre>	Method untuk mencocokkan token saat ini (tokens[currentToken]) dengan simbol terminal yang diharapkan. Jika cocok, node terminal baru dibentuk dengan nilai lexeme token tersebut, lalu currentToken dimajukan satu posisi. Jika tidak cocok, pointer received dan nilai expected diperbarui untuk keperluan pelaporan error, dan method mengembalikan nullptr.
	<pre>programNode() programHeader() declarationPart() constDeclaration() constantNode() typeDeclaration() varDeclaration() identifierList() typeName() arrayType() rangeNode() enumerated() recordType() fieldList() fieldPart() subprogramDeclaration() procedureDeclaration() functionDeclaration() blockNode() formalParameterList() parameterGroup() compoundStatement() statementList() statementNode() variableNode() componentVariable() indexList() assignmentStatement() ifStatement() caseStatement() caseBlock()</pre>	Method recursive descent parser untuk menangani masing-masing non-terminal pada grammar. Setiap method akan memeriksa kesesuaian susunan token terhadap aturan grammar dan membentuk bagian parse tree yang sesuai. Method mengembalikan std::shared_ptr<TreeNode> apabila parsing berhasil dan nullptr apabila terjadi syntax error.

	whileStatement() repeatStatement() forStatement() procedureFunctionCall() parameterList() expressionNode() simpleExpression() termNode() factorNode() relationalOperator() additiveOperator() multiplicativeOperator()	
TreeNode	TreeNode(Symbol node)	Konstruktur untuk membuat node parse tree tanpa nilai lexeme. Digunakan untuk node non-terminal atau terminal yang tidak membawa nilai literal (misalnya keyword dan operator).
	TreeNode(Symbol node, std::string value)	Konstruktur untuk membuat node parse tree beserta nilai lexeme token. Digunakan untuk terminal yang membawa nilai, seperti ident, intcon, realcon, string, dan charcon.
	Symbol nodeType	Menyimpan jenis simbol yang direpresentasikan node ini, baik terminal (misalnya Symbol::ident) maupun non-terminal (misalnya Symbol::PROGRAM). Nilai ini digunakan saat mencetak pohon melalui fungsi toString().
	std::string value	Menyimpan nilai lexeme token apabila node merepresentasikan terminal yang memiliki nilai literal. Kosong untuk keyword, operator, dan semua node non-terminal.
	std::vector<std::shared_ptr<TreeNode>> children	Menyimpan daftar child node dari suatu node dalam parse tree. Urutan child mencerminkan urutan kemunculan simbol dalam aturan grammar yang bersangkutan.
	void addChild(std::shared_ptr<TreeNode> newNode)	Menambahkan satu child node ke akhir daftar children. Dipanggil oleh setiap method parser saat menyusun struktur parse tree.
	bool outputTree(std::string destFile)	Membuka file output pada path destFile lalu memanggil printNode() secara rekursif dari root dengan kedalaman 0. Mengembalikan true

		apabila berhasil, atau false apabila file tidak dapat dibuka.
	<code>bool printNode(std::ofstream& file, int depth)</code>	Method rekursif yang mencetak satu node beserta seluruh child-nya ke dalam file. Setiap level kedalaman dicetak dengan awalan <code> </code> untuk indentasi dan <code>├─</code> tepat sebelum nama node, sehingga menghasilkan representasi pohon berbasis teks. Nilai lexeme turut dicetak dalam kurung apabila node adalah terminal yang memiliki nilai.
Token	<code>Token(Symbol symbol, int line)</code>	Konstruktur token tanpa nilai lexeme. Digunakan untuk keyword dan operator yang cukup diidentifikasi dari tipe simbolnya saja (misalnya <code>beginsy</code> , <code>plus</code> , <code>semicolon</code>). Parameter <code>line</code> menyimpan nomor baris kemunculan token di file sumber.
	<code>Token(Symbol symbol, std::string value, int line)</code>	Konstruktur token beserta nilai lexeme. Digunakan untuk token yang membawa nilai literal seperti <code>ident</code> , <code>intcon</code> , <code>realcon</code> , <code>string</code> , <code>charcon</code> , dan <code>comment</code> .
	<code>Symbol tokenType</code>	Menyimpan jenis token hasil lexical analysis sebagai nilai enum <code>Symbol</code> . Digunakan oleh parser untuk mencocokkan token dengan aturan grammar.
	<code>std::string value;</code>	Menyimpan isi lexeme token. Bernilai kosong untuk token tanpa nilai literal (keyword dan operator).
	<code>int line</code>	Menyimpan nomor baris di file sumber tempat token ini ditemukan. Digunakan oleh parser untuk melaporkan lokasi syntax error.
	<code>static const std::map<std::string , Symbol> keywordMap;</code>	Struktur mapping dari representasi string token (misalnya <code>"beginsy"</code> , <code>"ident"</code>) ke nilai enum <code>Symbol</code> yang bersesuaian. Digunakan oleh <code>readTokens()</code> saat membaca file token dan oleh lexer saat menghasilkan objek <code>Token</code> .
	<code>static std::vector<Token> readTokens(std::stri ng filepath)</code>	Method statis yang membaca file token baris per baris dan mengonversi setiap baris menjadi objek <code>Token</code> menggunakan <code>keywordMap</code> . Token dengan nilai lexeme (seperti <code>ident(x)</code>) diparse dengan mengekstrak nama token dan

		nilai dalam kurung secara terpisah. Digunakan pada mode operasi 2 di main.cpp (input berupa file token, bukan kode sumber).
--	--	---

3.5. Alur Kerja Program



Gambar 3.5.1 Alur Kerja Program

Secara umum, program bekerja melalui dua tahap utama, yaitu proses lexical analysis menggunakan DFA untuk mengubah source code menjadi token, kemudian proses parsing untuk membentuk parse tree. Komponen utama yang digunakan meliputi Lexer, DFA, Transition, dan Parser. Berikut penjelasan alur program secara keseluruhan:

1. Inisialisasi DFA

Pada tahap ini, program membaca file aturan DFA (dfa_rules.txt). Semua aturan transisi akan dimuat ke dalam struktur transition.

2. Inisialisasi Lexer

Pada tahap ini, lexer membuka file source code, kemudian membaca karakter per karakter.

3. Preprocessing

Lexer akan melewati spasi, tab, newline, sekaligus melakukan update nomor baris. Dilakukan supaya DFA hanya memproses karakter yang relevan.

4. Proses Tokenisasi

Merupakan loop utama dari program. Menggunakan fungsi tokenize() yang didefinisikan di lexer.cpp. Dengan alur sebagai berikut:

- a) Mulai dari
 - currentState = start_state
 - lexeme = ""
- b) Untuk setiap karakter,

Ambil next state:
nextState = getNextState(...)

- c) Terdapat tiga kemungkinan kasus:

CASE 1: Transisi Valid

Lanjut membangun token. Ini mencakup transisi spesifik dan wildcard.

```
lexeme += currentChar;  
currentState = nextState;
```

CASE 2: Masuk Dead State

Terjadi apabila tidak ada transisi spesifik dan tidak ada wildcard.

```
if(DFA.isFinalState(currentState))
```

- o Jika sebelumnya final:
 - simpan token
 - reset state
- o Jika tidak:
 - error / stop

- d) EOF Handling

Jika masih ada lexeme dan state final, maka tetap jadi token.

- e) Output

Menghasilkan vector<Token>.

5. Parsing

Setelah proses tokenisasi selesai, vector token dikirim ke parser. Parser menggunakan metode recursive descent parsing untuk memeriksa kesesuaian token terhadap grammar bahasa. Apabila ditemukan token yang tidak sesuai dengan grammar, parser akan mengembalikan nullptr dan menampilkan informasi syntax error berupa token yang diharapkan, token yang diterima, serta nomor baris error.

6. Pembentukan Parse Tree

Selama proses parsing, parser membangun struktur parse tree menggunakan TreeNode. Setiap non-terminal grammar direpresentasikan sebagai node yang memiliki child sesuai hasil parsing produksi grammar. Parse tree hanya akan dihasilkan apabila seluruh proses parsing berhasil dan source code valid secara sintaks.

3.6. Pendekatan Implementasi

Implementasi parser menggunakan metode Recursive Descent Parsing yang bersifat top-down untuk membangun parse tree secara sistematis sesuai dengan tata bahasa yang didefinisikan. Setiap aturan produksi dalam

grammar dipetakan menjadi satu fungsi spesifik (seperti `programNode()`, `declarationPart()`, dan `assignmentStatement()`), yang mempermudah pelacakan alur sintaksis dan pemeliharaan kode. Untuk menangani ambiguitas dan memastikan ketepatan pemilihan aturan, diterapkan mekanisme *backtracking* dengan menyimpan indeks token saat ini (`currentToken`) sebelum mencoba suatu produksi. Jika produksi tersebut gagal (mengembalikan `nullptr`), posisi token akan dikembalikan ke titik awal sehingga parser dapat mencoba jalur alternatif.

Selain itu, parser dilengkapi dengan sistem pelaporan kesalahan yang informatif yang mencatat simbol yang diharapkan (`expected`) dibandingkan dengan simbol yang diterima (`received`) beserta nomor barisnya, guna mempermudah proses debugging bagi pengguna. Struktur data berbasis pohon (`TreeNode`) dengan smart pointers (`std::shared_ptr`) dipilih untuk mengelola hierarki hasil analisis secara efisien dan aman tanpa risiko kebocoran memori, sehingga memungkinkan visualisasi struktur program yang jelas melalui output pohon yang terindentasi.

BAB 4

HASIL IMPLEMENTASI

4.1. Kode Program

Class Token:

```
1 class Token {
2     private:
3         Symbol tokenType;
4         std::string value;
5         int line;
6     public:
7         static const std::map<std::string, Symbol> keywordMap;
8         static std::vector<Token> readTokens(std::string filepath);
9         Token(Symbol symbol, int line) : tokenType(symbol), line(line){}
10        Token(Symbol symbol, std::string value, int line) : tokenType(symbol), value(value), line(line){}
11        int getLine() const { return line; }
12        Symbol getTokenType() const { return tokenType; }
13        std::string getValue() const { return value; }
14    };
```

Penjelasan:

Class Token merepresentasikan satu unit leksikal yang dihasilkan oleh lexical analyzer. Setiap token menyimpan tiga informasi:

1. tokenType: jenis token, bertipe Symbol (enum yang membedakan ident, intcon, plus, casesy, dll.)
 2. value: nilai aktual token sebagai string (misalnya "42", "myVar", "Hello")
 3. line: nomor baris token di kode sumber, berguna untuk pesan error
- Dua konstruktor: satu tanpa value (untuk token yang nilainya tidak relevan, seperti operator atau keyword), dan satu dengan value (untuk identifier, konstanta, string). keywordMap adalah peta statis yang memetakan string keyword ke Symbol-nya, digunakan lexer untuk membedakan ident biasa dari keyword seperti begin, end, case.
 - readTokens(): entry point lexer yang membaca file sumber dan menghasilkan vector of tokens yang kemudian dikonsumsi parser.

Class Parser:

```
1  class Parser {
2      private:
3          std::vector<Token> tokens;
4          int currentToken;
5          Token *received;
6          Symbol expected;
7      public:
8          Parser(std::vector<Token> tokens) : tokens(tokens), currentToken(0) {}
9          std::shared_ptr<TreeNode> buildTree();
10         std::shared_ptr<TreeNode> terminal(Symbol symbol);
11
12         std::shared_ptr<TreeNode> programNode();
13         std::shared_ptr<TreeNode> programHeader();
14         std::shared_ptr<TreeNode> declarationPart();
15         std::shared_ptr<TreeNode> constDeclaration();
16         std::shared_ptr<TreeNode> constantNode();
17         std::shared_ptr<TreeNode> typeDeclaration();
18         std::shared_ptr<TreeNode> varDeclaration();
19         std::shared_ptr<TreeNode> identifierList();
20         std::shared_ptr<TreeNode> typeNode();
21         std::shared_ptr<TreeNode> arrayType();
22         std::shared_ptr<TreeNode> rangeNode();
23         std::shared_ptr<TreeNode> enumerated();
24         std::shared_ptr<TreeNode> recordType();
25         std::shared_ptr<TreeNode> fieldList();
26         std::shared_ptr<TreeNode> fieldPart();
27         std::shared_ptr<TreeNode> subprogramDeclaration();
28         std::shared_ptr<TreeNode> procedureDeclaration();
29         std::shared_ptr<TreeNode> functionDeclaration();
30         std::shared_ptr<TreeNode> blockNode();
31         std::shared_ptr<TreeNode> formalParameterList();
32         std::shared_ptr<TreeNode> parameterGroup();
33         std::shared_ptr<TreeNode> compoundStatement();
34         std::shared_ptr<TreeNode> statementList();
35         std::shared_ptr<TreeNode> statementNode();
36         std::shared_ptr<TreeNode> variableNode();
37         std::shared_ptr<TreeNode> componentVariable();
38         std::shared_ptr<TreeNode> indexList();
39         std::shared_ptr<TreeNode> assignmentStatement();
40         std::shared_ptr<TreeNode> ifStatement();
41         std::shared_ptr<TreeNode> caseStatement();
42         std::shared_ptr<TreeNode> caseBlock();
43         std::shared_ptr<TreeNode> whileStatement();
44         std::shared_ptr<TreeNode> repeatStatement();
45         std::shared_ptr<TreeNode> forStatement();
46         std::shared_ptr<TreeNode> procedureFunctionCall();
47         std::shared_ptr<TreeNode> parameterList();
48         std::shared_ptr<TreeNode> expressionNode();
49         std::shared_ptr<TreeNode> simpleExpression();
50         std::shared_ptr<TreeNode> termNode();
51         std::shared_ptr<TreeNode> factorNode();
52         std::shared_ptr<TreeNode> relationalOperator();
53         std::shared_ptr<TreeNode> additiveOperator();
54         std::shared_ptr<TreeNode> multiplicativeOperator();
55     };
```

```

1  std::shared_ptr<TreeNode> Parser::buildTree(){
2      std::shared_ptr<TreeNode> tree = programNode();
3      if(tree == nullptr){
4          std::cout << "Parsing failed," << std::endl;
5          std::cout << "          expected: " << toString(expected) << std::endl;
6          std::cout << "          received: " << toString(received->getTokenType()) << std::endl;
7          std::cout << "          at line " << received->getLine() << std::endl;
8      }
9      return tree;
10 }
11
12 std::shared_ptr<TreeNode> Parser::terminal(Symbol symbol){
13     std::shared_ptr<TreeNode> ptr = nullptr;
14     if(tokens[currentToken].getTokenType() == symbol){
15         ptr = std::make_shared<TreeNode>(symbol);
16         ptr->setValue(tokens[currentToken].getValue());
17         currentToken++;
18     } else {
19         received = &tokens[currentToken];
20         expected = symbol;
21     }
22     return ptr;
23 }

```

Penjelasan:

Class Parser adalah inti dari komponen syntax analysis. Data privatnya terdiri dari:

1. tokens: vector semua token hasil lexer, tidak berubah selama parsing
 2. currentToken: indeks token yang sedang diproses, bergerak maju saat terminal() berhasil match
 3. received dan expected: menyimpan token yang diterima dan yang diharapkan saat terjadi mismatch, untuk keperluan pesan error di buildTree()
- Konstruktor menerima vector tokens dan menginisialisasi currentToken = 0.
 - Fungsi buildTree() adalah entry point parser — memanggil programNode() (simbol awal grammar), dan jika hasilnya nullptr mencetak pesan error yang informatif: token apa yang diterima, token apa yang diharapkan, dan di baris berapa.
 - Fungsi terminal(Symbol symbol) adalah helper fundamental yang dipakai oleh semua fungsi grammar. Cara kerjanya: jika token saat ini cocok dengan symbol yang diminta, dia membuat leaf node baru, mengisi value-nya dari token, lalu increment currentToken dan return node tersebut. Jika tidak cocok, dia mencatat token yang salah ke received dan expected, lalu return nullptr. Ini yang menjadi dasar dari pola backtracking di seluruh parser.
 - Semua fungsi lainnya (dari programNode() sampai multiplicativeOperator()) masing-masing mengimplementasikan satu aturan produksi grammar — persis seperti yang sudah kita bahas di fungsi caseBlock() dan caseStatement(). Setiap fungsi membuat node non-terminal, memanggil kombinasi terminal() dan fungsi grammar lain sesuai aturan produksinya, lalu mengembalikan node yang sudah terisi children atau nullptr jika parsing gagal.

Class TreeNode:

```
1  class TreeNode {
2      private:
3          Symbol nodeType;
4          std::string value;
5          std::vector<std::shared_ptr<TreeNode>> children;
6      public:
7          TreeNode(Symbol node) : nodeType(node){}
8          TreeNode(Symbol node, std::string value) : nodeType(node), value(value){}
9          Symbol getNodeType() const { return nodeType; }
10         std::vector<std::shared_ptr<TreeNode>> getChildren() const { return children; }
11         std::string getValue() const { return value; }
12         void setValue(std::string value) { this->value = value; }
13         void addChild(std::shared_ptr<TreeNode> newNode);
14         bool outputTree(std::string destFile) const;
15         bool printNode(std::ofstream& file, int depth) const;
16     };
```

```
1  void TreeNode::addChild(std::shared_ptr<TreeNode> newNode){
2      children.push_back(newNode);
3  }
4
5  bool TreeNode::outputTree(std::string destFile) const {
6      std::ofstream output(destFile);
7      if(!output){
8          cerr << "Failed to open output file: " << destFile << endl;
9          return false;
10     }
11     return printNode(output, 0);
12 }
13
14 bool TreeNode::printNode(std::ofstream& output, int depth) const {
15     for(int i = 0; i < depth; i++){
16         if(i == depth - 1){
17             output << "└- ";
18         }
19         else{
20             output << "| ";
21         }
22     }
23     output << toString(this->nodeType);
24     if(isToken(this->nodeType) && value != ""){
25         output << "(" << value << ")";
26     }
27     output << std::endl;
28     for(const auto& node : children){
29         node->printNode(output, depth+1);
30     }
31     return true;
32 }
```

Penjelasan:

Class `TreeNode` adalah node dalam parse tree. Setiap node menyimpan:

1. `nodeType`: tipe simbol node ini (bisa non-terminal seperti `CASE_BLOCK`, atau terminal seperti `casesy`)
 2. `value`: nilai aktual untuk node terminal (misalnya isi identifier atau konstanta)
 3. `children`: daftar child nodes, menggunakan `vector<shared_ptr<TreeNode>>` agar memori dikelola otomatis
- Dua konstruktor tersedia: satu tanpa `value` (untuk non-terminal dan terminal tanpa nilai khusus), satu dengan `value` (untuk terminal yang menyimpan nilai konkret seperti `ident("myVar")`).
 - Fungsi `addChild()` hanya memanggil `push_back` ke vector `children`. Fungsi `outputTree()` membuka file output lalu mendelegasikan ke `printNode()`.
 - Fungsi `printNode()` mencetak tree secara rekursif, tiap level kedalaman dicetak dengan prefix `|` untuk bukan-anak-terakhir dan `|—` untuk anak terakhir, menghasilkan tampilan tree teks seperti yang kamu lihat di contoh parse tree. Setelah mencetak dirinya sendiri, dia loop ke seluruh `children` dan memanggil `printNode(output, depth+1)` pada masing-masing — inilah rekursi yang membangun keseluruhan representasi visual tree.

4.2. Aturan Grammar

1. `<program> -> program-header + declaration-part + compound-statement + period`
2. `<program-header> -> programsy + ident + semicolon`
3. `<declaration-part> -> (const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*`
4. `<const-declaration> -> constsy + (ident + eql + constant + semicolon)+`
5. `<constant> -> charcon | string | [(plus | minus)? + (ident | intcon | realcon)]`
6. `<type-declaration> -> typesy + (ident + eql + type + semicolon)+`
7. `<var-declaration> -> varsy + (identifier-list + colon + type + semicolon)+`
8. `<identifier-list> -> ident (comma + ident)*`
9. `<type> -> ident | array-type | range | enumerated | record-type`
10. `<array-type> -> arraysy + lbrack + (range | ident) + rbrack + ofsy + type`
11. `<range> -> constant + period + period + constant`
12. `<enumerated> -> lparent + ident + (comma + ident)* + rparent`
13. `<record-type> -> recordsy + field-list + endsy`
14. `<field-list> -> field-part + (semicolon + field-part)*`
15. `<field-part> -> identifier-list + colon + type`
16. `<subprogram-declaration> -> procedure-declaration | function-declaration`

- 17.<procedure-declaration> -> proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon
- 18.<function-declaration> -> functionsy + ident + (formal-parameter-list)? + colon + ident + semicolon + block + semicolon
- 19.<block> -> declaration-part + compound-statement
- 20.<formal-parameter-list> -> lparent + parameter-group + (semicolon + parameter-group)* + rparent
- 21.<parameter-group> -> identifier-list + colon + (ident | array-type)
- 22.<compound-statement> -> beginsy + statement-list + endsy
- 23.<statement-list> -> statement (semicolon + statement)*
- 24.<statement> -> (assignment-statement | if-statement | case-statement | while-statement | repeat-statement | for-statement | procedure/function-call)?
- 25.<variable> -> ident + (component-variable)*
- 26.<component-variable> -> (lbrack + index-list + rbrack) | (period + ident)
- 27.<index-list> -> (intcon | charcon | ident) + (comma + index-list)*
- 28.<assignment-statement> -> variable + becomes + expression
- 29.<if-statement> -> ifsy + expression + thensy + statement + (elsy + statement)?
- 30.<case-statement> -> casesy + expression + ofsy + case-block + endsy
- 31.<case-block> -> constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*
- 32.<while-statement> -> whilesy + expression + dosy + **compound-statement** + semicolon
- 33.<repeat-statement> -> repeatsy + statement-list + untilsy + expression
- 34.<for-statement> -> forsy + ident + becomes + expression + (tosy | downtosy) + expression + dosy + **compound-statement**
- 35.<procedure/function-call> -> ident + (lparent + parameter-list? + rparent)
- 36.<parameter-list> -> expression (comma + expression)*
- 37.<expression> -> simple-expression (relational-operator + simple-expression)?
- 38.<simple-expression> -> (plus | minus)? term (additive-operator + term)*
- 39.<term> -> factor (multiplicative-operator + factor)*
- 40.<factor> -> ident | intcon | realcon | charcon | string | (lparent + expression + rparent) | (notsy + factor) | procedure/function-call | variable
- 41.<relational-operator> -> eql | neq | gtr | geq | lss | leq
- 42.<additive-operator> -> plus | minus | orsy
- 43.<multiplicative-operator> -> times | rdiv | idiv | imod | andsy

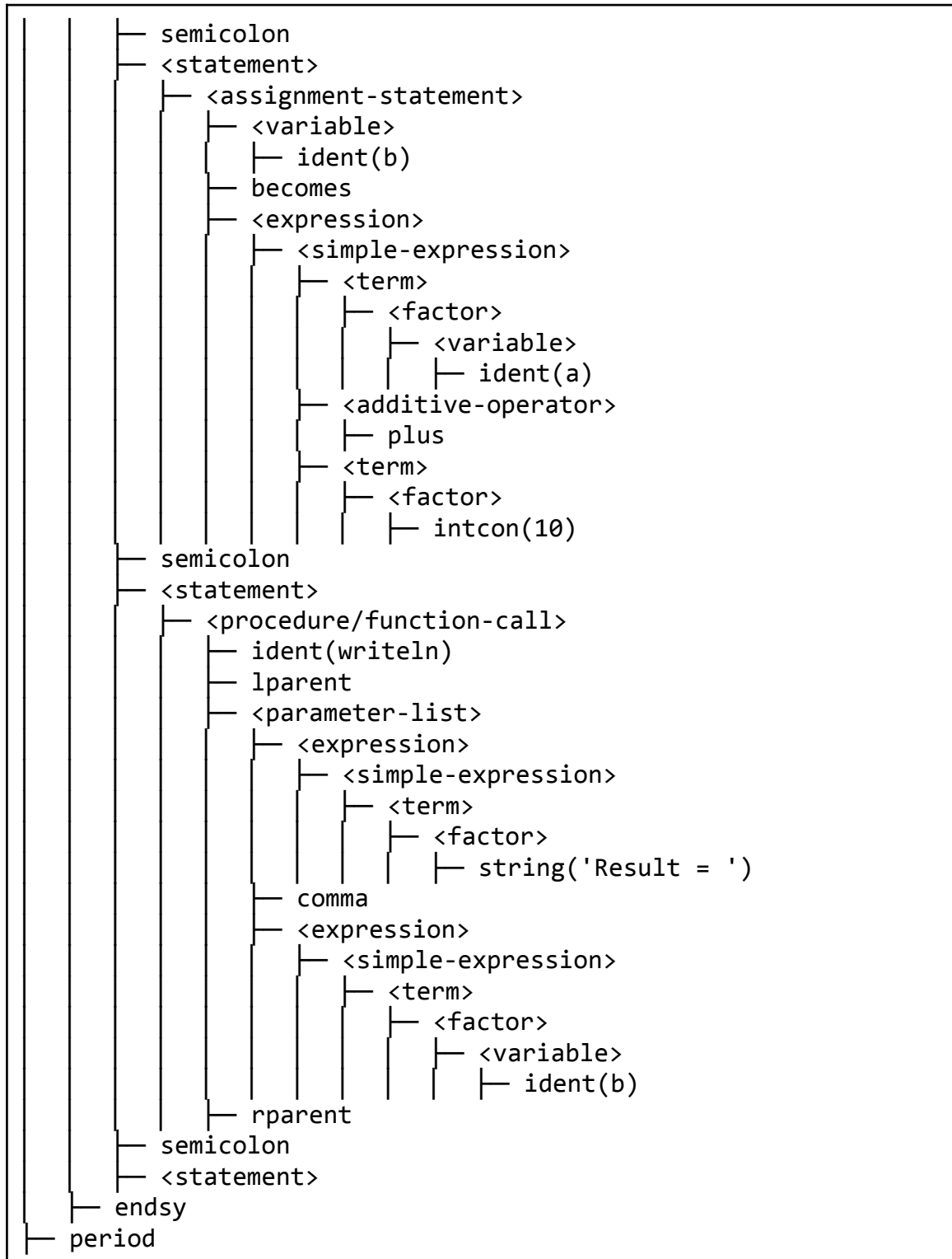
BAB 5

EKSPERIMEN

❖ Kasus uji 1

Tabel 5.1 Kasus Uji 1

Input
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Output
<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(Hello) │ └── semicolon ├─ <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ ├── ident(a) │ │ │ ├── comma │ │ │ └── ident(b) │ │ ├── colon │ │ ├── <type> │ │ │ └── ident(integer) │ │ └── semicolon │ └─ <compound-statement> │ ├── beginsy │ └─ <statement-list> │ └─ <statement> │ └─ <assignment-statement> │ ├── <variable> │ │ └── ident(a) │ ├── becomes │ └─ <expression> │ ├── <simple-expression> │ │ └─ <term> │ │ └─ <factor> │ │ └─ intcon(5)</pre>



❖ Kasus uji 2

Tabel 5.2 Kasus Uji 2

Input

```

program RepeatCaseTest;
var
  x, choice: integer;
begin
  x := 5;
  repeat
    x := x - 1
  until x == 0;
  choice := 2;
  case choice of
    1: writeln('one');
    2: writeln('two');
    3, 4: writeln('three or four')
  end
end.

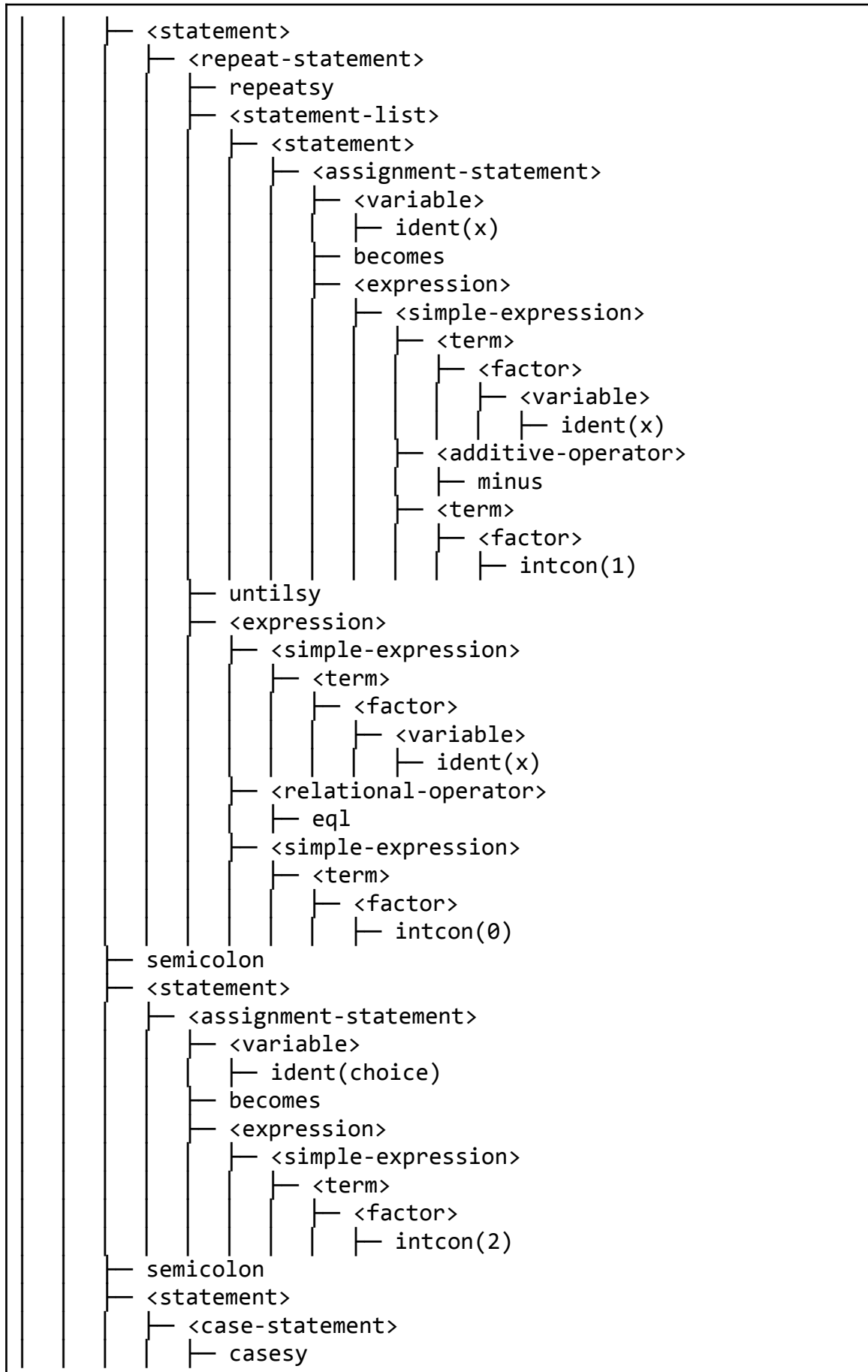
```

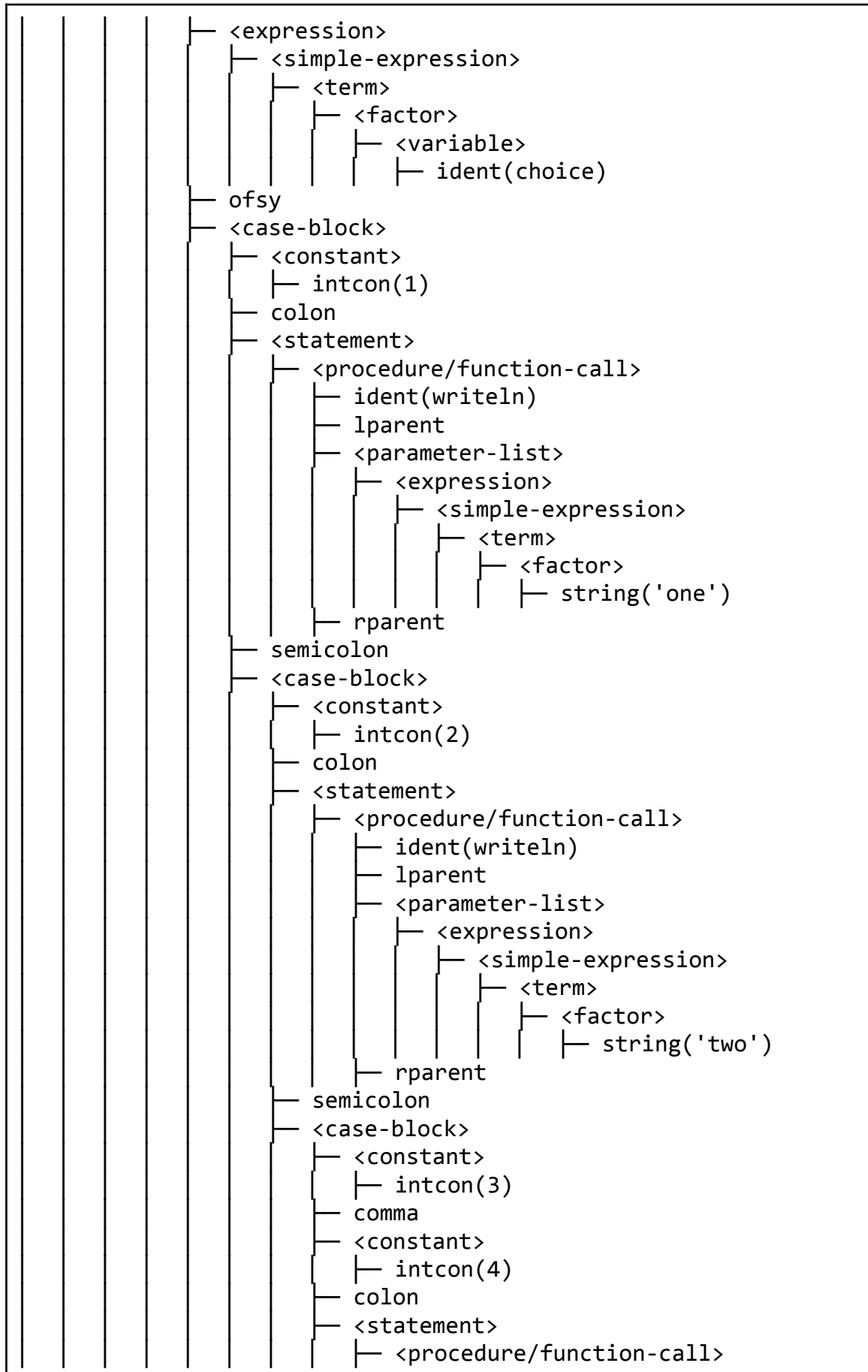
Output

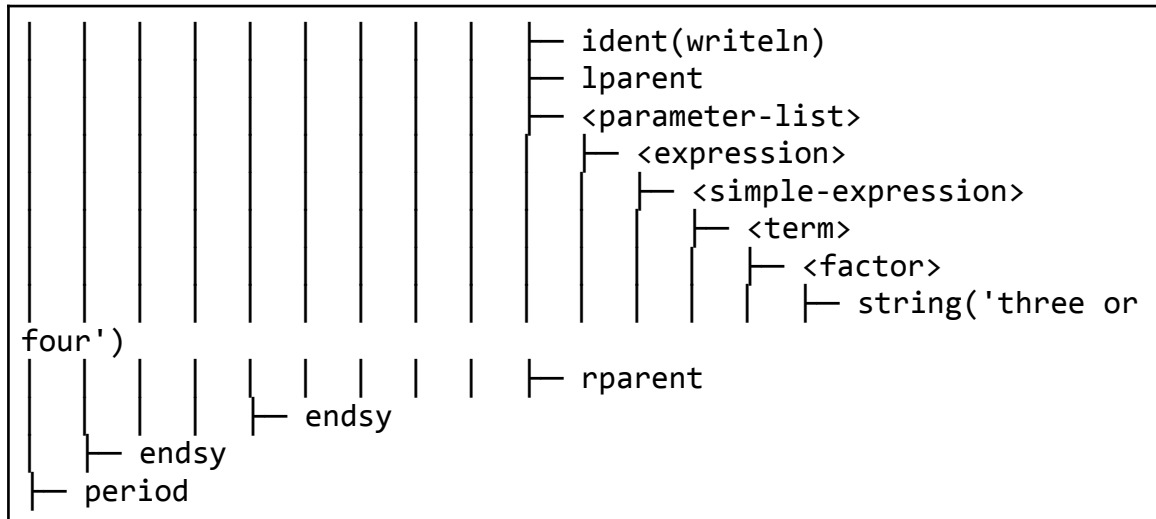
```

<program>
├─ <program-header>
│   ├── programsy
│   ├── ident(RepeatCaseTest)
│   └── semicolon
├─ <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   ├── ident(x)
│   │   │   ├── comma
│   │   │   └── ident(choice)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident(integer)
│   │   └── semicolon
│   └─ <compound-statement>
│       ├── beginsy
│       └─ <statement-list>
│           ├── <statement>
│           │   ├── <assignment-statement>
│           │   │   ├── <variable>
│           │   │   │   ├── ident(x)
│           │   │   │   ├── becomes
│           │   │   │   └─ <expression>
│           │   │   │       ├── <simple-expression>
│           │   │   │       │   ├── <term>
│           │   │   │       │   │   ├── <factor>
│           │   │   │       │   │   └─ intcon(5)
│           │   └─ semicolon

```



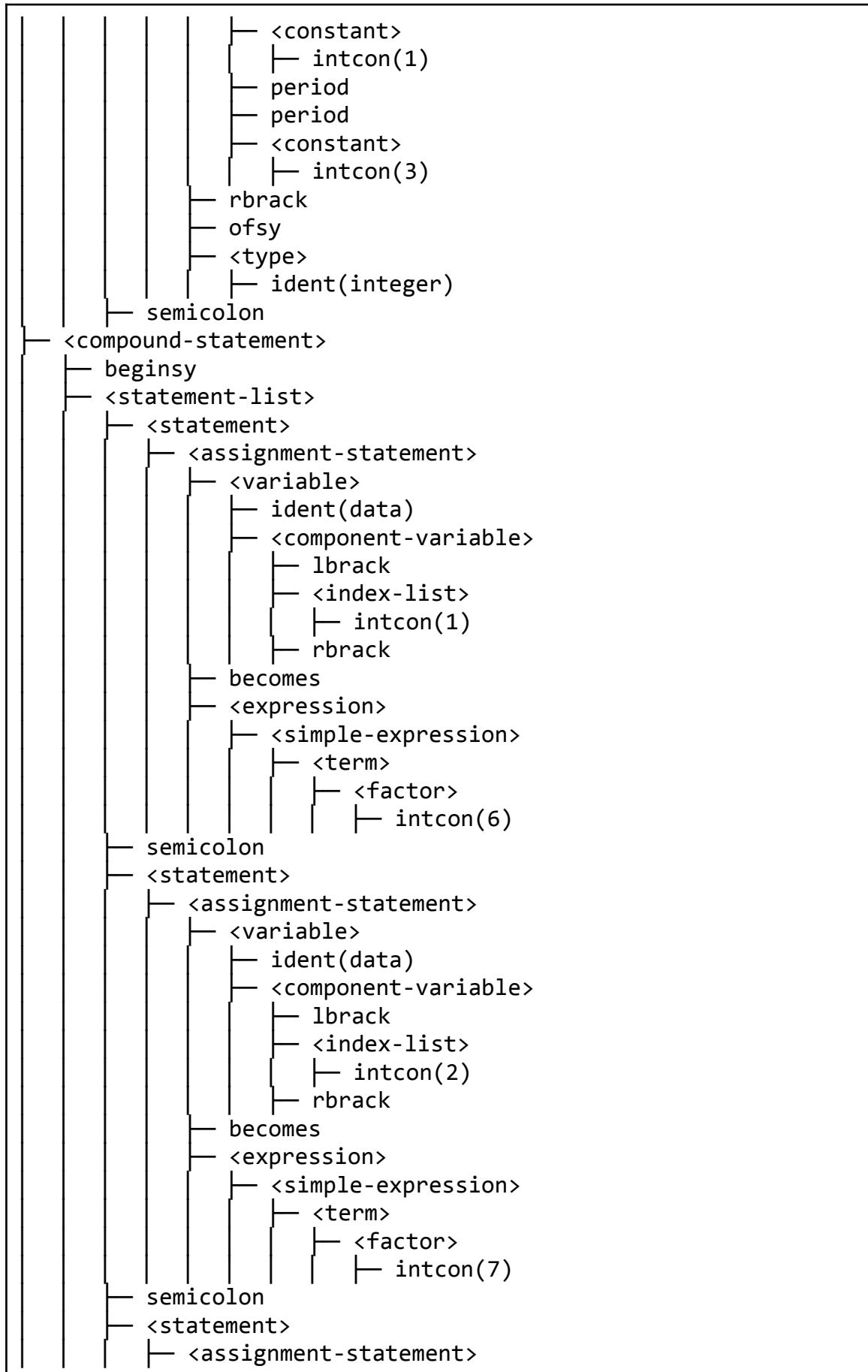


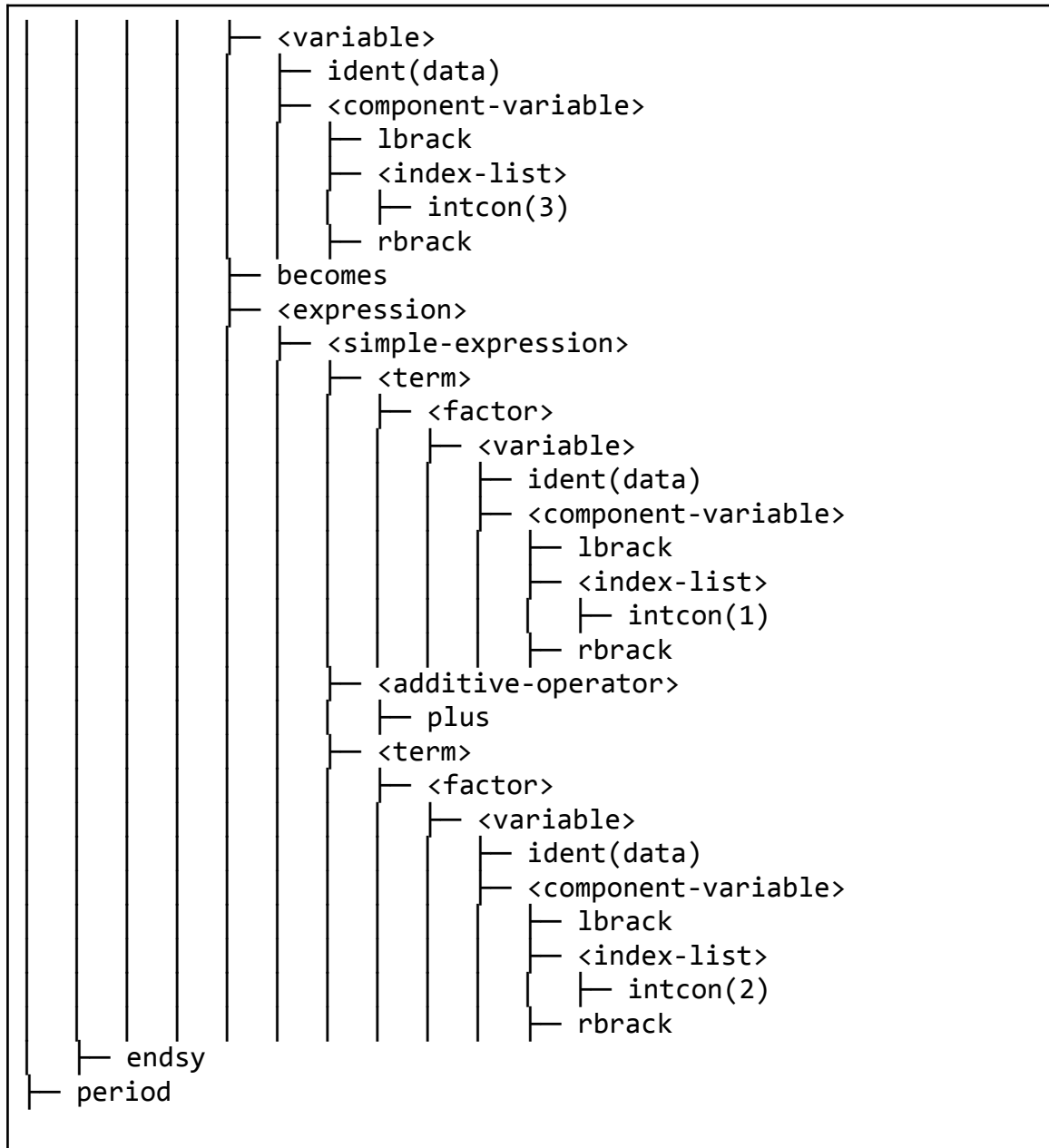


❖ Kasus uji 3

Tabel 5.3 Kasus Uji 3

Input
<pre> program TestRangeArray; var data: array[1..3] of integer; begin data[1] := 6; data[2] := 7; data[3] := data[1] + data[2] end. </pre>
Output
<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(TestRangeArray) │ └── semicolon ├─ <declaration-part> │ └─ <var-declaration> │ ├── varsy │ ├── <identifier-list> │ │ └─ ident(data) │ ├── colon │ └─ <type> │ └─ <array-type> │ ├── arraysy │ ├── lbrack │ └─ <range> </pre>





❖ Kasus uji 4

Tabel 5.4 Kasus Uji 4

Input
<pre> program Test4; type Season == (Spring, Summer, Autumn, Winter); var s: integer; begin s := 1; </pre>

```

case s of
  1: writeln(s);
  2: writeln(s)
end
end.

```

Output

```

<program>
├─ <program-header>
│   ├── programsy
│   ├── ident(Test4)
│   └── semicolon
├─ <declaration-part>
│   ├── <type-declaration>
│   │   ├── typesy
│   │   ├── ident(Season)
│   │   ├── eql
│   │   └── <type>
│   │       ├── <enumerated>
│   │       │   ├── lpARENT
│   │       │   ├── ident(Spring)
│   │       │   ├── comma
│   │       │   ├── ident(Summer)
│   │       │   ├── comma
│   │       │   ├── ident(Autumn)
│   │       │   ├── comma
│   │       │   ├── ident(Winter)
│   │       │   └── rpARENT
│   │       └── semicolon
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident(s)
│       ├── colon
│       ├── <type>
│       │   └── ident(integer)
│       └── semicolon
└─ <compound-statement>
    ├── beginsy
    └── <statement-list>
        └── <statement>
            └── <assignment-statement>
                ├── <variable>
                │   └── ident(s)
                ├── becomes
                ├── <expression>
                └── <simple-expression>

```


		endsy
		period

❖ Kasus uji 5

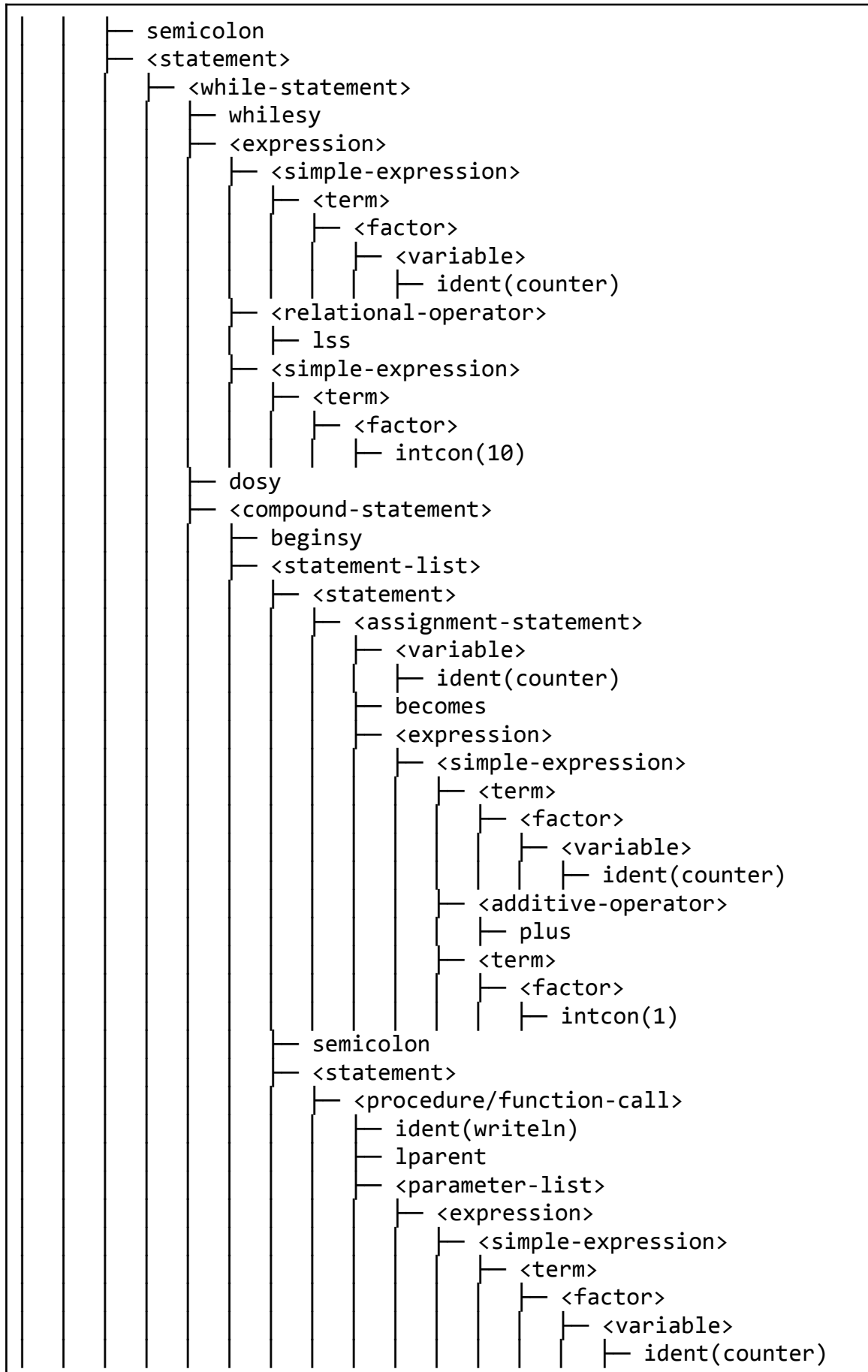
Tabel 5.5 Kasus Uji 5

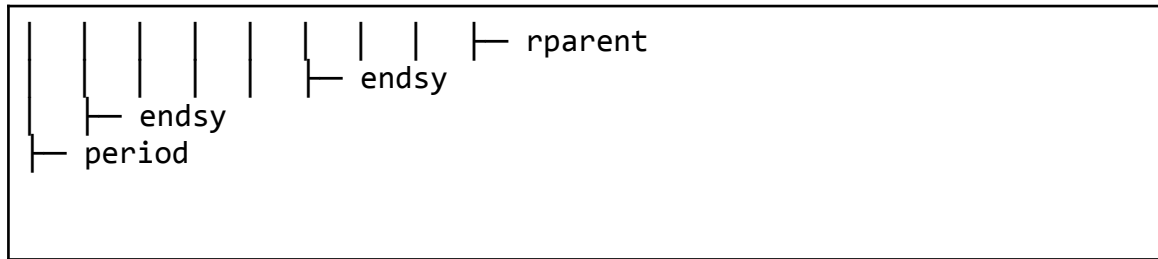
Input
<pre> program TestDownTo; var i: integer; begin for i := 5 downto 1 do begin writeln(i); writeln(67) end end. </pre>
Output
<pre> <program> <program-header> programsy ident(TestDownTo) semicolon <declaration-part> <var-declaration> varsy <identifier-list> ident(i) colon <type> ident(integer) semicolon <compound-statement> beginsy <statement-list> <statement> <for-statement> forsy ident(i) becomes <expression> <simple-expression> </pre>

❖ Kasus uji 6

Tabel 5.6 Kasus Uji 6

Input
<pre> program TestWhile; var counter: integer; begin counter := 0; while counter < 10 do begin counter := counter + 1; writeln(counter) end; end. </pre>
Output
<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(TestWhile) │ └── semicolon ├─ <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ └── ident(counter) │ │ ├── colon │ │ ├── <type> │ │ │ └── ident(integer) │ │ └── semicolon │ └─ <compound-statement> │ ├── beginsy │ └─ <statement-list> │ └─ <statement> │ └─ <assignment-statement> │ ├── <variable> │ │ └── ident(counter) │ ├── becomes │ └─ <expression> │ ├── <simple-expression> │ │ └─ <term> │ │ └─ <factor> │ │ └─ intcon(0) </pre>





❖ Kasus uji 7 (error code)

Tabel 5.7 Kasus Uji 7

Input
<pre> program Hello; var a, b: integer; begin a := 5 b := a + 10; writeln('Result = ', b); end. </pre>
Output
<pre> Parsing failed, expected: endsy received: ident at line 6 </pre>

BAB 6

KESIMPULAN DAN SARAN

6.1. Kesimpulan

Pada milestone ini telah berhasil diimplementasikan sebuah parser untuk bahasa Arion menggunakan metode Recursive Descent Parsing yang bersifat top-down. Parser menerima aliran token hasil lexical analysis dari milestone sebelumnya dan memeriksa kesesuaiannya terhadap 43 aturan grammar yang telah didefinisikan. Setiap aturan produksi dipetakan langsung menjadi sebuah fungsi dalam kode, sehingga struktur implementasi mencerminkan secara langsung hierarki grammar bahasa Arion.

Selain validasi sintaks, parser juga berhasil membangun struktur Parse Tree secara lengkap menggunakan class `TreeNode` yang dikelola melalui `std::shared_ptr`, sehingga manajemen memori berjalan secara otomatis tanpa risiko kebocoran. Parse Tree yang dihasilkan memuat seluruh simbol terminal dan non-terminal sesuai dengan derivasi grammar, dan dapat divisualisasikan dalam bentuk teks terindentasi melalui fungsi `outputTree()`. Sistem pelaporan kesalahan yang terintegrasi pada fungsi `buildTree()` mampu menampilkan token yang diharapkan, token yang diterima, beserta nomor baris terjadinya kesalahan, sehingga proses debugging menjadi lebih mudah.

6.2. Saran

Untuk pengembangan selanjutnya, sistem pelaporan error dapat ditingkatkan lebih lanjut. Saat ini parser hanya melaporkan satu titik kegagalan pertama yang ditemukan, lalu berhenti. Teknik error recovery seperti panic mode atau phrase-level recovery dapat diterapkan agar parser mampu melanjutkan analisis setelah menemukan error, sehingga seluruh kesalahan sintaks dalam satu file dapat dilaporkan sekaligus dalam satu kali eksekusi. Hal ini akan meningkatkan kenyamanan pengguna secara signifikan.

Selain itu, Parse Tree yang dihasilkan pada milestone ini masih memuat semua node termasuk yang tidak memiliki kontribusi semantik, seperti tanda kurung, koma, dan kata kunci struktural. Pada tahap selanjutnya, Parse Tree ini dapat dipangkas menjadi Abstract Syntax Tree (AST) dengan menghilangkan node-node redundan tersebut, sehingga representasi yang dihasilkan lebih ringkas dan efisien untuk diproses pada tahap semantic analysis dan byte code generation.

LAMPIRAN

Repository GitHub: <https://github.com/SyaqinaOctavia/MMD-Tubes-IF2224-2026.git>

NIM	Kontribusi Tugas	Persentase
13524040	Implementasi Grammar, Testing program, Testcase input-output,	25%
13524042	Penulisan laporan, Implementasi Grammar	25%
13524048	Class Parser, TreeNode, Symbol, Token. Implementasi Grammar	25%
13524088	Penulisan laporan, Implementasi Grammar	25%

REFERENSI

John E., Rajeev M., dan Jeffrey D. *Introduction to Automata Theory, Languages, and Computation 3rd Edition*.
<https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613-Introduction-to-Automata-Theory-Languages.-and-Computation-Edit-ion-3.pdf>

Ruslan Spivak. *Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees*. Diakses 7 Mei 2026.
<https://ruslanspivak.com/lbasi-part7>

GeeksForGeeks. *Introduction to Syntax Analysis in Compiler Design*. Diakses 7 Mei 2026.
<https://www.geeksforgeeks.org/compiler-design/introduction-to-syntax-analysis-in-compiler-design/>